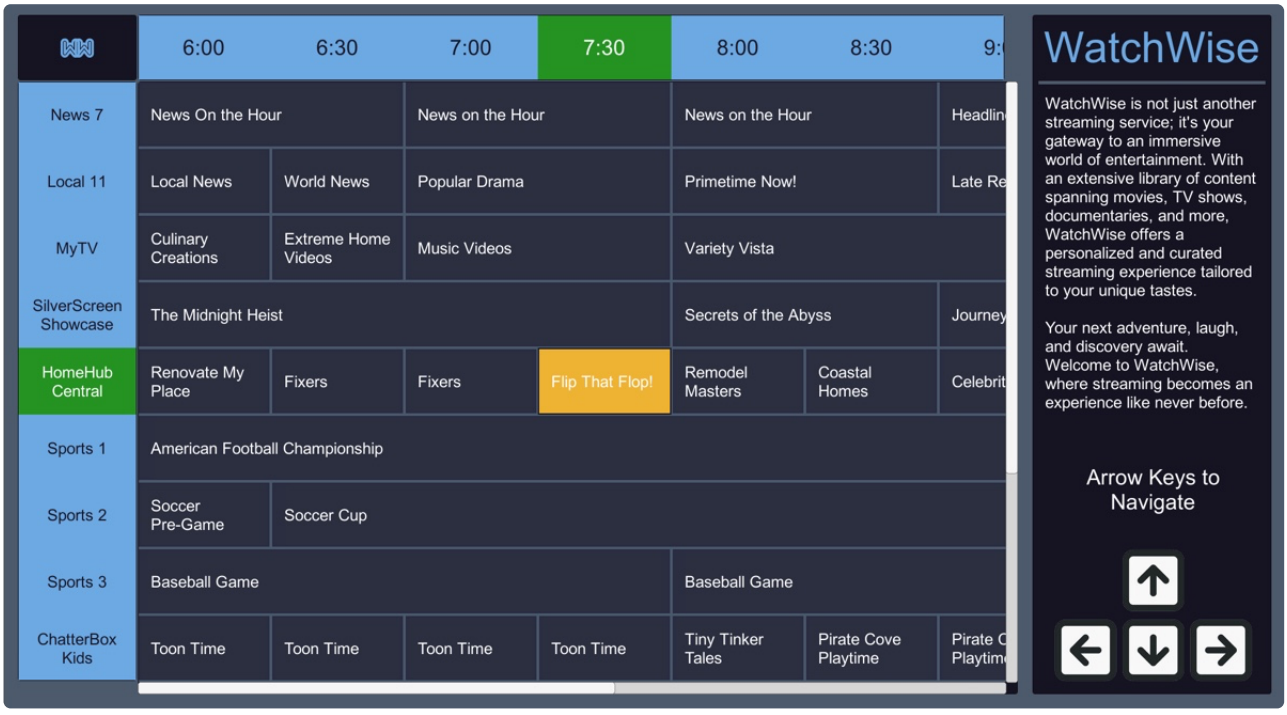


User Manual

EnhancedGrid can help you display your lists, grids, and tables in Unity.

EnhancedGrid is a data-driven MVC (Model - View - Controller) solution for displaying one or two dimensional UI layouts in the Unity game engine. It optionally uses object pooling to reduce the memory and processor demands of your project, making it more efficient than Unity's built-in scroller.

Jumping, looping, cell expansion, multiple cell types, view driven cell sizes, selection, and more are all supported.



Streaming platform schedule grid included with the plugin

created with craft

User Manual Contents

Quick Start

You can find videos on how to use EnhancedGrid on our

Overview

EnhancedGrid can help you display your lists, grids, and

MVC

What is MVC?

MVC stands for Model, View,

Coordinates

EnhancedGrid does not use the same y coordinate system that

Flow and Layout

EnhancedGrid is set up to view data as a serialized, one-dimensional list. This allows it to

Recycling

EnhancedGrid has the option to recycle cells into an object pool.

Filling & Looping

EnhancedGrid has four repeat modes which can be set on a per-axis basis. So you can have

Jumping

Jumping is when you move to a different position or cell in a

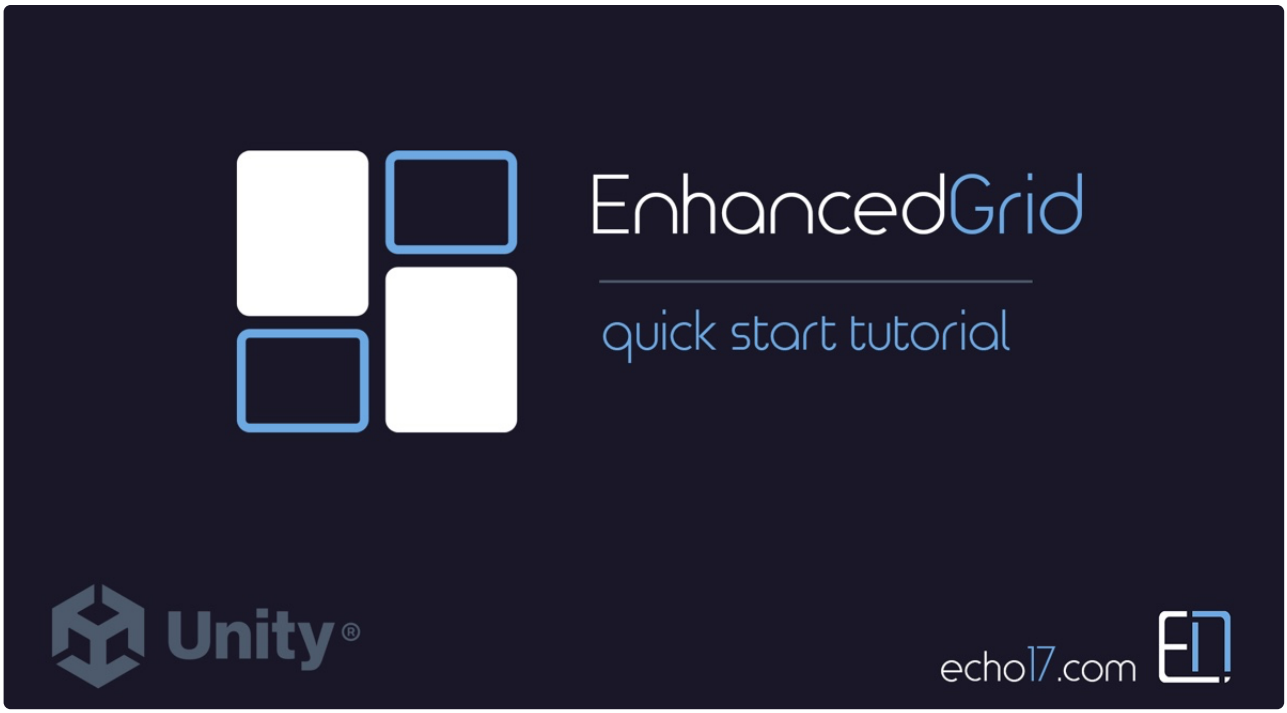
Snapping

EnhancedGrid has additional components that you can attach,

created with craft

Quick Start

> You can find videos on how to use EnhancedGrid on our Youtube channel
>
> youtube.com/@echo17software



Steps

Please view the quick start video in the Documentation folder or on our Youtube channel for more details on the following.

1. Create a ScrollView in your scene and attach the EnhancedGrid component.
2. Create a view script in your project and have it implement the IEnhancedGridCell interface. Set up a method that will receive data from your controller. This method will update your cell's view state. Create a UI prefab in your project and attach the view script.
3. Create a controller script in your project and have it implement the IEnhancedGridDelegate interface. Attach the script to a game object in your scene. Link the grid object created in step 1. Link the cell prefab created in step 2. Call InitializeGrid one time on the grid object, most likely in your script's Start method. Fill in the IEnhancedGridDelegate methods in your controller script.
4. Call RecalculateGrid any time you change your data or add new data.

created with craft

↑

Overview

EnhancedGrid can help you display your lists, grids, and tables in Unity.

EnhancedGrid is a data-driven MVC (Model - View - Controller) solution for displaying one or two dimensional UI layouts in the Unity game engine. It optionally uses object pooling to reduce the memory and processor demands of your project, making it more efficient than Unity's built-in scroller.



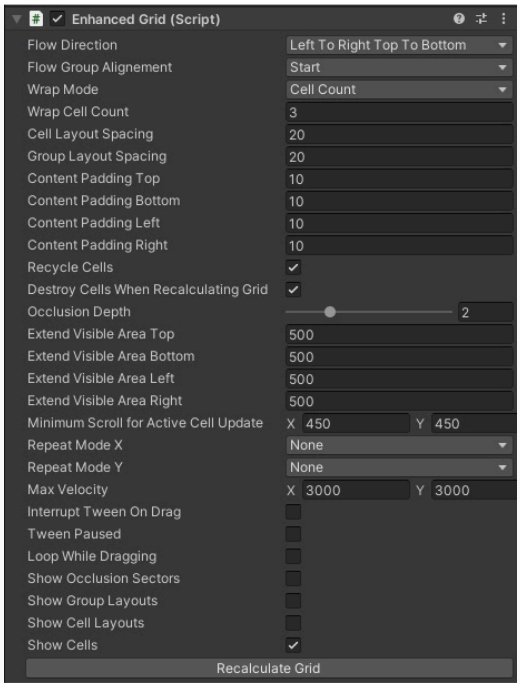
Inventory example included with the plugin

How EnhancedGrid Works

Your projects utilizing EnhancedGrid will need at least one controller script per grid. However, your controller can be linked to multiple grids, one of the many advatanges of the MVC design pattern.

You use EnhancedGrid by setting up a link to your grid(s) in the controller script. A one time call to initialized the grid is required, usually done in the Start method of your controller. Once you have your model set up and the IEnhancedGridDelegate interface methods implemented, you can call RecalculateGrid on the EnhancedGrid object, which will then build out your grid.

created with craft



EnhancedGrid component inspector

Flow Direction: How to layout the cells in your grid. Any direction starting with LeftToRight or RightToLeft will create groups of cells in rows. Any direction starting with TopToBottom or BottomToTop will create groups of cells in columns. See the Flow and Layout section for more details.

Flow Group Alignment: How the cells inside a group (row or column) are aligned and expanded. See the Flow and Layout section for more details.

Wrap Mode: How the grid will wrap cells using the flow direction. Wrapping can be none (making a one-dimensional list of items), cell count, or size. See the Flow and Layout section for more details.

Wrap Cell Count: If Wrap Mode is set to Cell Count, then this will be how many cells to create layouts for in a group (row or column). See the Flow and Layout section for more details.

Wrap Size: If Wrap Mode is set to Size, then this will be how many pixels to allocate to each group (row or column). See the Flow and Layout section for more details.

Cell Layout Spacing: The amount of pixels between each cell inside a group (row or column). See the Flow and Layout section for more details.

Group Layout Spacing: The amount of pixels between groups (row or column). See the Flow and Layout section for more details.

Content Padding: How much space to have surrounding the groups and cells inside the content game object.

Recycle Cells: Whether to recycle cells based on the viewable area, or just create all cells. See Recycling section for more details.

Destroy Cells When Recalculating Grid: Whether to completely recreate the cells if a RecalculateGrid method is called. This is only relevant if recycling since cells will always be destroyed and recreated when recycling is off.

created with craft

Occlusion Depth: How many layers of recursion to use when constructing the occlusion graph. See Recycling section for more details.

Extend Visible Area: How far outside the Viewport to extend the area where the grid will look for cells becoming visible or invisible. See Recycling section for more details.

Minimum Scroll For Active Cell Update: How far the grid can scroll before it checks for cell visibility. This value should be lower then the extend visible area to avoid cells "popping" into visibility. See the Recycling section for more details.

Repeat Mode: Whether to fill the content area with copies of the cells. If overlapping is used, then copies will be made beyond the content boundaries if the cells' sizes exceed those parameters. If no overlapping is used, then the grid will only make copies if the cells can fit inside the content boundaries. If loop is used, then the grid will make enough copies of the cells to give the illusion of an infinite set of cells. See the Filling & Looping section for more details.

Max Velocity: The maximum speed the grid can travel in each axis. This is useful in some conditions to eliminate extreme jumps in position when the loop jump occurs to give the illusion of an infinite list.

Interrupt Tween On Drag: Whether to stop tweening from a jump or snap if the pointer (mouse or finger) starts to drag the grid.

Tween Paused: Whether to stop the tweeing from a jump or snap. You can also call the ToggleTweenPaused method if you want to resume tweening from the current scroll position instead of where the tween left off when paused.

Loop While Dragging: Whether to do a loop jump while the pointer (mouse or finger) is dragging the grid. This is generally not recommended as it can cause erratic jumps. See the Filling & Looping section for more details.

Show Occlusion Sectors: Debug setting to visually show where the occlusion sector boundaries are. Not recommended for use outside of running tests.

Show Group Layouts: Debug setting to visually show where the group (row or column) layout boundaries are. Not recommended for use outside of running tests.

Show Cell Layouts: Debug setting to visually show where the cell layout boundaries are. Not recommended for use outside of running tests.

Show Cells: Shows the cells in the grid. This should be on unless you are debugging and need to get the cells out of the way to visualize other elements, like the group or cell layouts.

created with craft

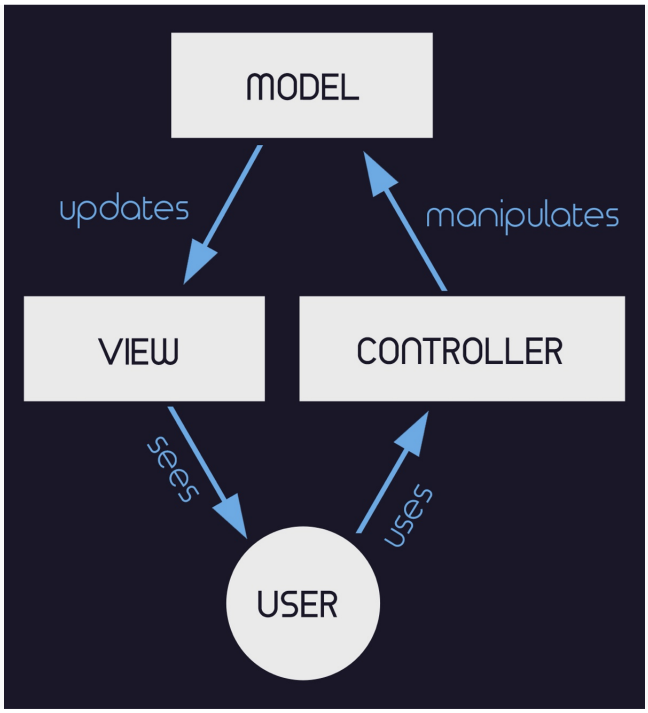
↑

MVC

What is MVC?

MVC stands for Model, View, Controller. It is a time-tested method of separating your view (what is actually displayed to the user) and your controller (the code that handles the logic and updates the data). The controller directly manipulates the model (the data), which then drives the view.

EnhancedGrid acts as part of the view in the MVC design pattern, requesting information from the controller and updating its cells by referencing the model. This lets you focus more on the data and the logic of the grid, rather than trying to manipulate the view elements directly.



MVC Diagram. Source: Wikipedia

Controller

The controller script must inherit from the IEnhancedGridDelegate interface, which requires four callbacks that the grid will invoke:

GetCellCount: Tells the grid how many cells to create.

GetCellPrefab: Tells the grid what prefab to use for a cell at a given index.

GetCellProperties: Tells the grid the cell size and expansion weight at a given index.

UpdateCell: Tells the controller that a cell view needs updating. This breaks the MVC pattern somewhat, but gives the developer more controller on how the cell is updated. Here you would typically call some public method on your cell view, passing the model of the cell to the view so that it can update its contents.

created with craft

View

Cell view scripts must implement the IEnhancedGridCell interface, which requires three properties and two methods:

Properties:

DataIndex (int): The serialized data index of the cell. See note about Data Index in the Model section.

RepeatIndex (int): The actual index in the grid. If looping and / or filling is turned off, the RepeatIndex is the same as the DataIndex. If looping or filling is turned on, then the RepeatIndex is index of the cell based on how many times the cell was copied to fill or loop the grid.

ContainerTransform (Transform): Each cell has a container transform which fits the cell layout created by the grid. Your cell view does not have to fit this container, but your cell view layout will be relative to the cell layout calculated by the grid.

Methods:

GetCellTypeIdentifier (string): Returns a unique string for a given cell view. In grids that have multiple cell types, each type should return a different string.

GetDrawPriority (float): Returns the draw order preference for the cell. Higher values are drawn last, which makes them on top of other cells. This is useful if your cell views overlap and you want to control which ones are viewed on top.

Model

EnhancedGrid does not actually need a model to work, but without one you would not have any useful information to display in your cells. That could be ideal if you are just looking to display cells at a particular location without needing any information in the cells themself.

Data Index

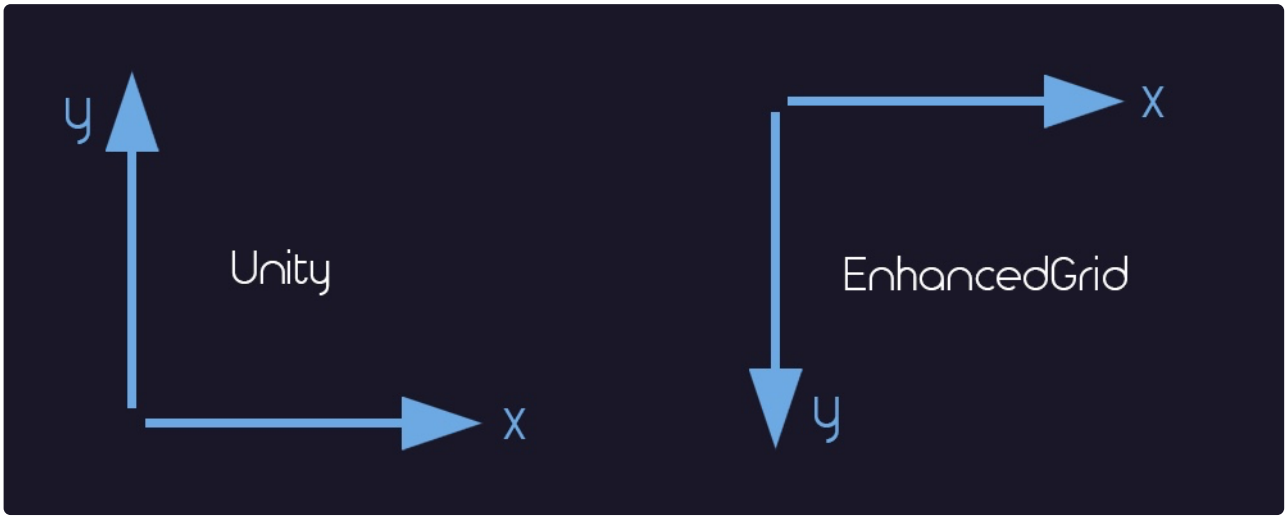
EnhancedGrid assumes your data is one dimensional, referencing each cell as an index in a list. It does not care how you store your model, or even if you have one, only that your controller can provide a total cell count, a cell prefab, and cell size information based on that one dimensional index, known as a data index. You can use a list, a two-dimensional array, a complex JSON structure, or nothing at all to store the model in your controller, so long as you can provide the cells with the appropriate model information for a particular data index. This may require some translation in your controller script to turn your model structure into a serialized one dimensional array.

In your controller's UpdateCell delegate method, you would typically call some public method on your cell view, passing the model of the cell to the view so that it can update its contents.

created with craft

Coordinates

EnhancedGrid does not use the same y coordinate system that Unity does for UI. Unity's UI y coordinates start at the bottom and go toward the top. EnhancedGrid uses a more natural system of starting the y coordinates at the top and going toward the bottom. This is more in line with how typical grid or list information is displayed in most UI systems.



Graphical representation of Unity and EnhancedGrid coordinate systems

created with craft

↑

Flow and Layout

EnhancedGrid is set up to view data as a serialized, one-dimensional list. This allows it to format the data in a number of ways without changing the underlying data.

> Note: you do not need to store your data in a one-dimensional list, but you will need to be able to translate your structure into a one-dimensional list in order to communicate with EnhancedGrid.

The way the elements are displayed is known as the flow direction. There are eight different ways the flow can be set. Each row or column of cells is known as a group.

Groups in **Rows**:

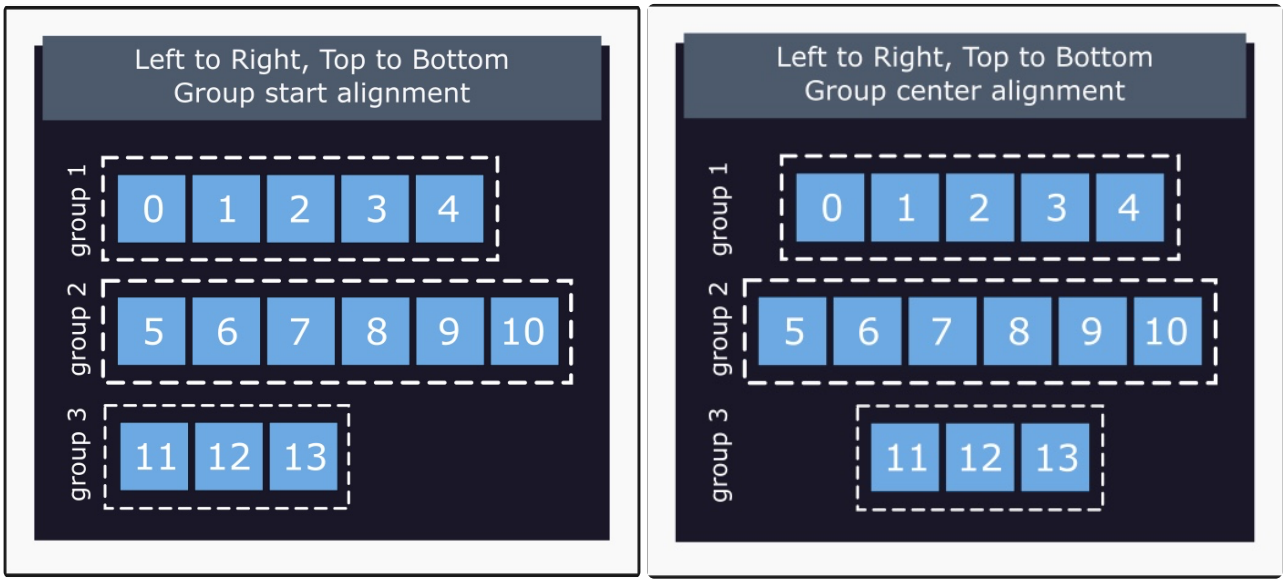
- Left to Right, Top to Bottom
- Right to Left, Top to Bottom
- Left to Right, Bottom to Top
- Right to Left, Bottom to Top

Groups in **Columns**:

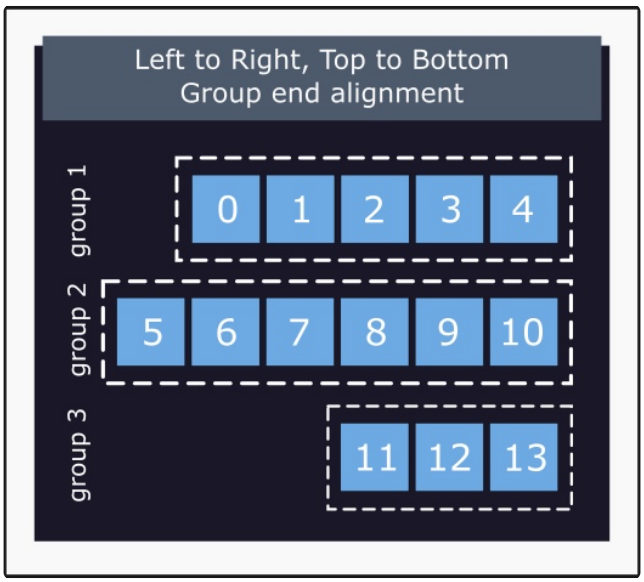
- Top to Bottom, Left to Right
- Top to Bottom, Right to Left
- Bottom to Top, Left to Right
- Bottom to Top, Right to Left

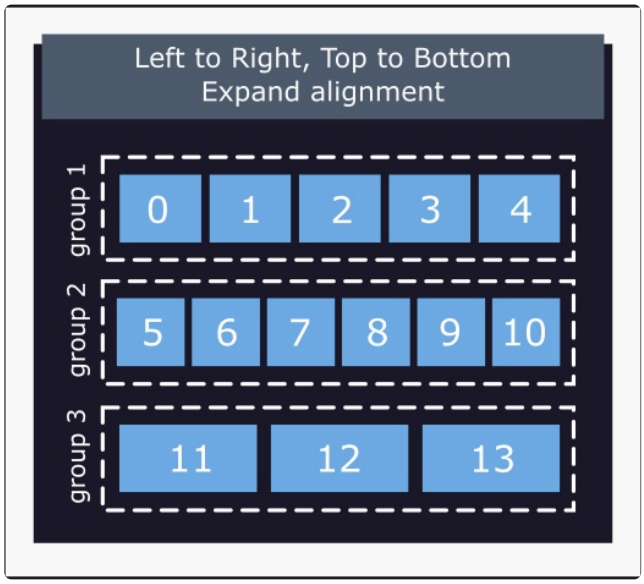
Cells within a group can be aligned in different ways to give unique effects.

Left to Right, Top to Bottom (Rows)

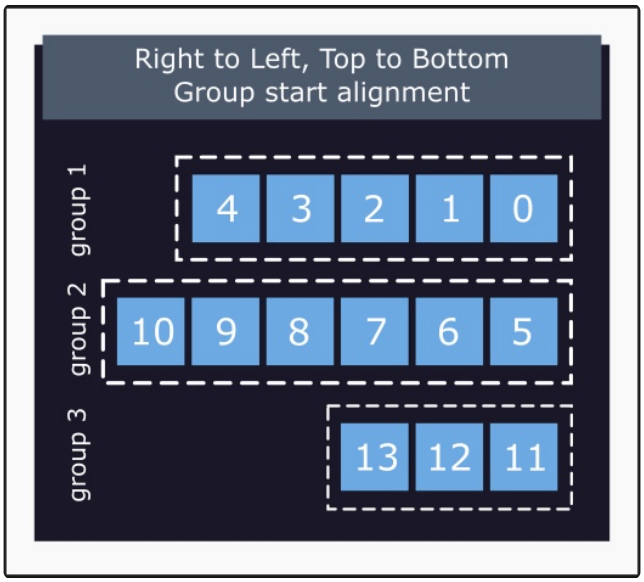


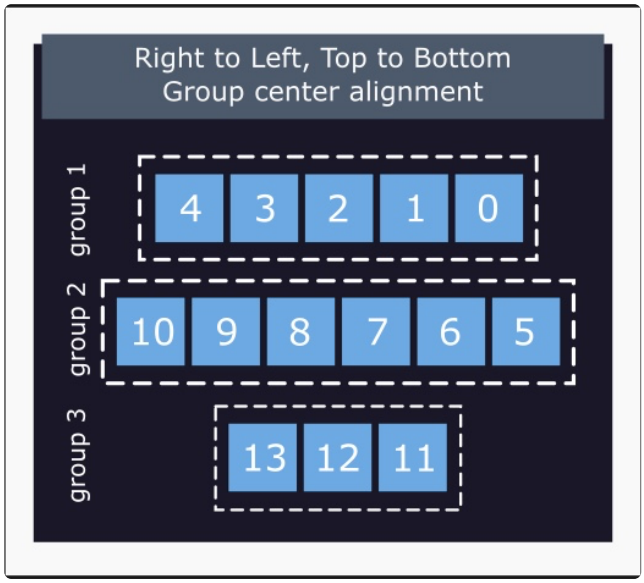
created with craft

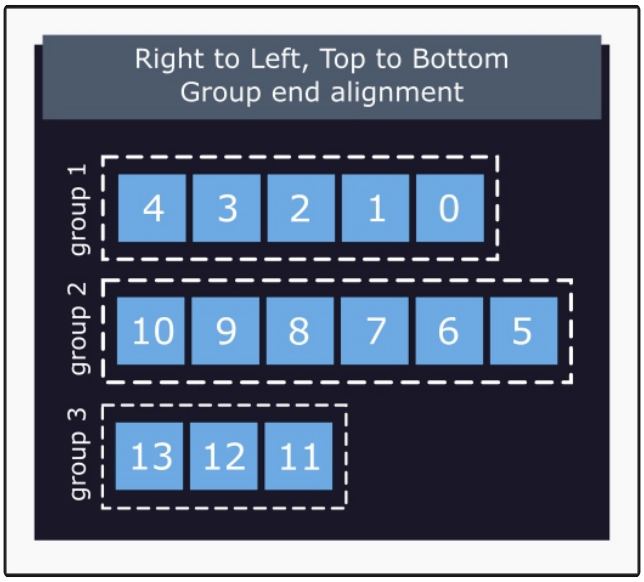


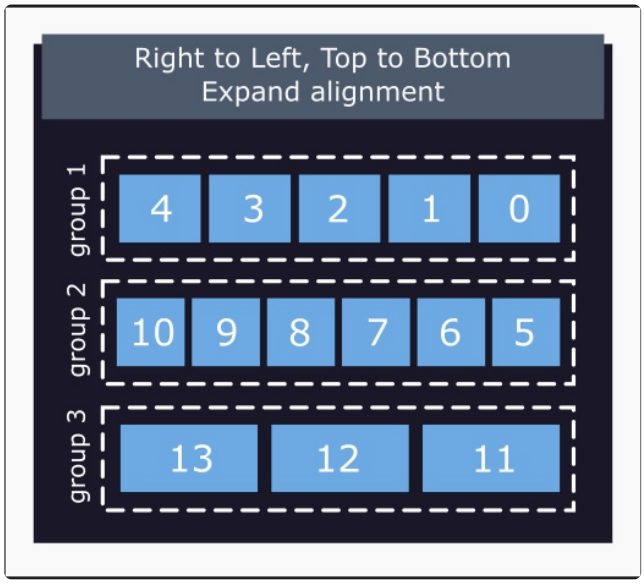


Right to Left, Top to Bottom (Rows)



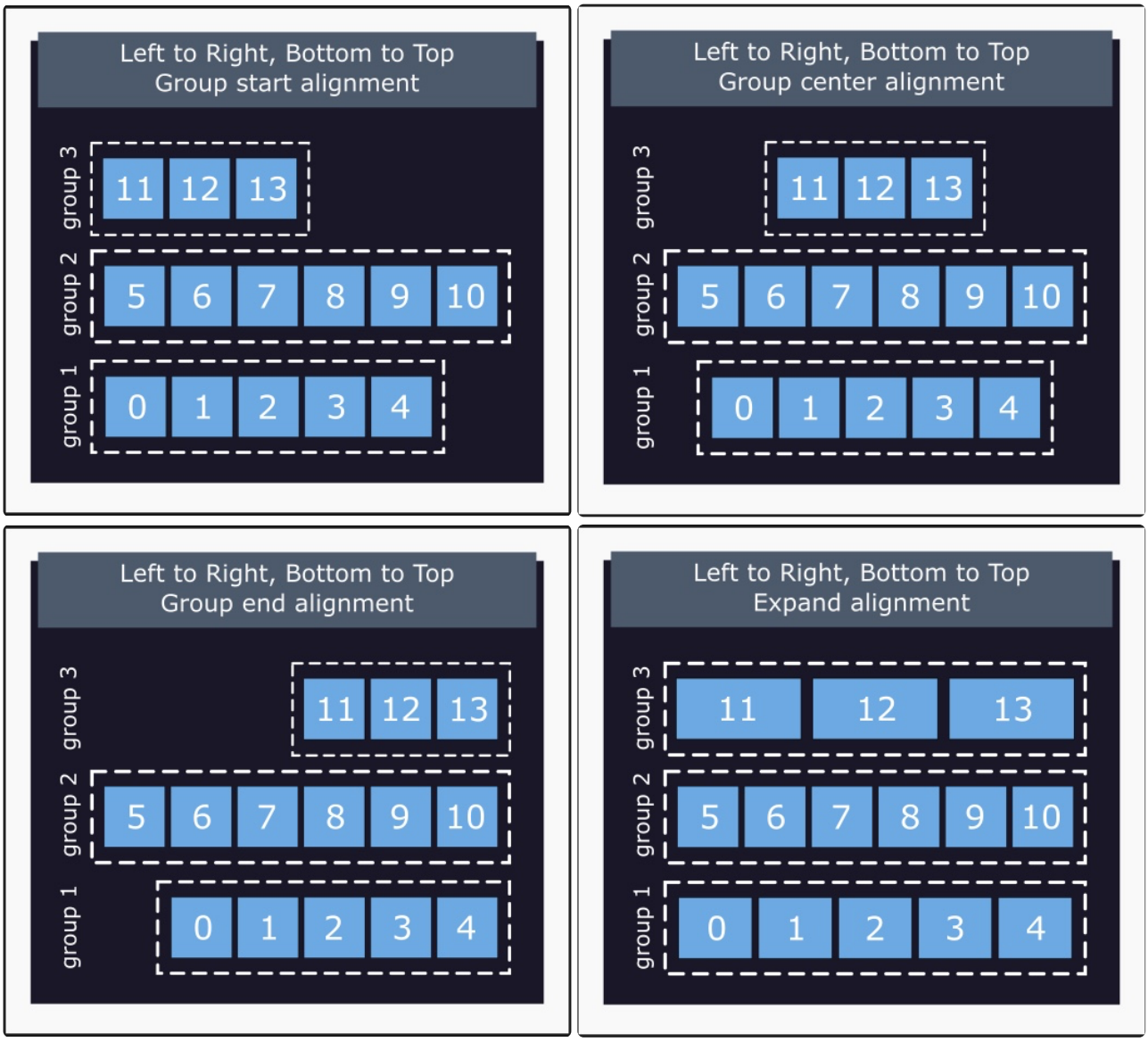






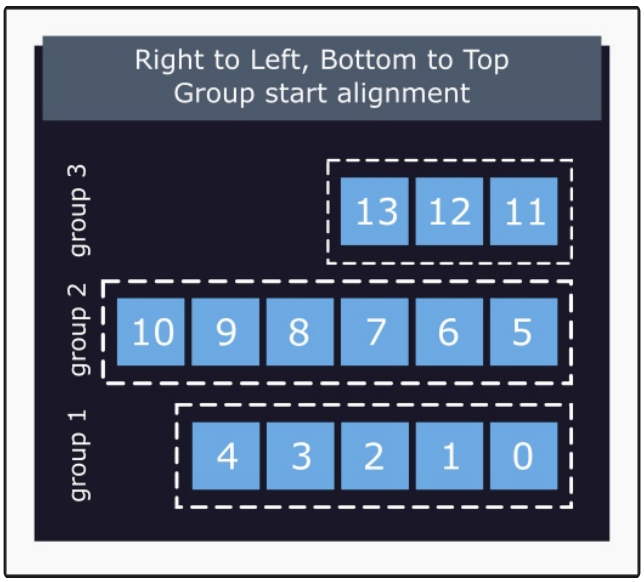
created with craft

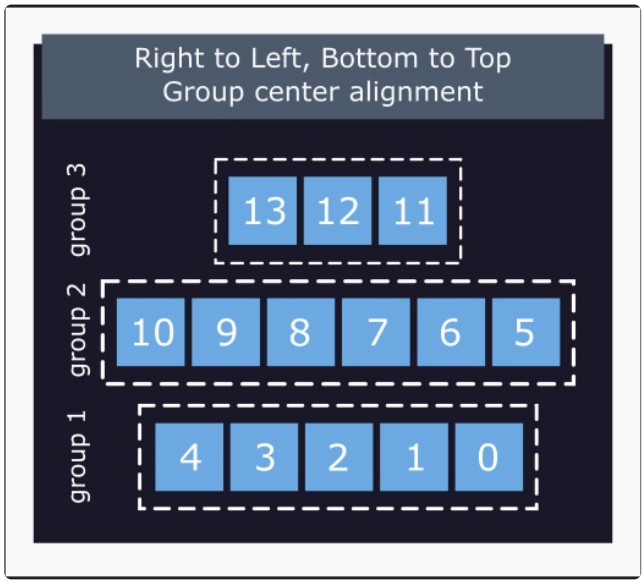
Left to Right, Bottom to Top (Rows)

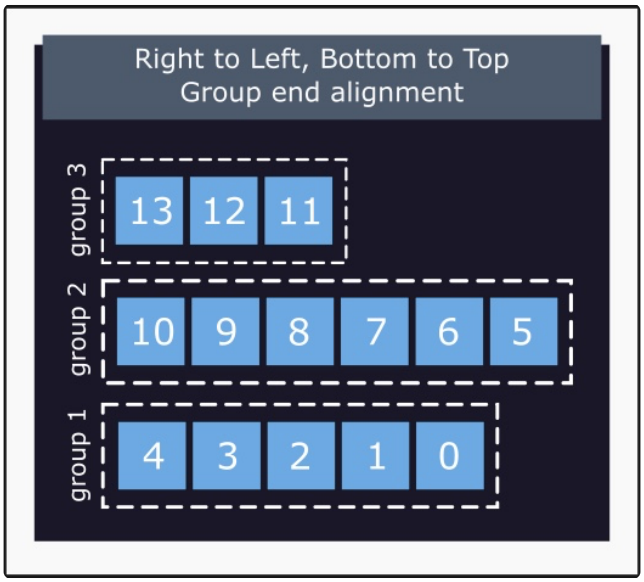


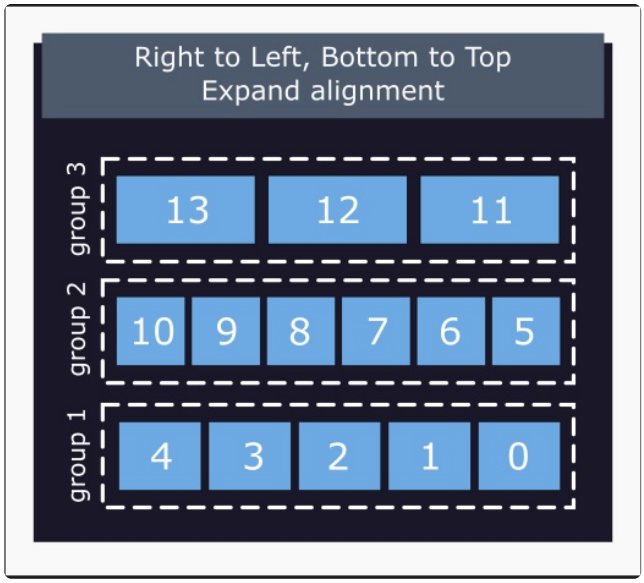
Right to Left, Bottom to Top (Rows)

created with craft

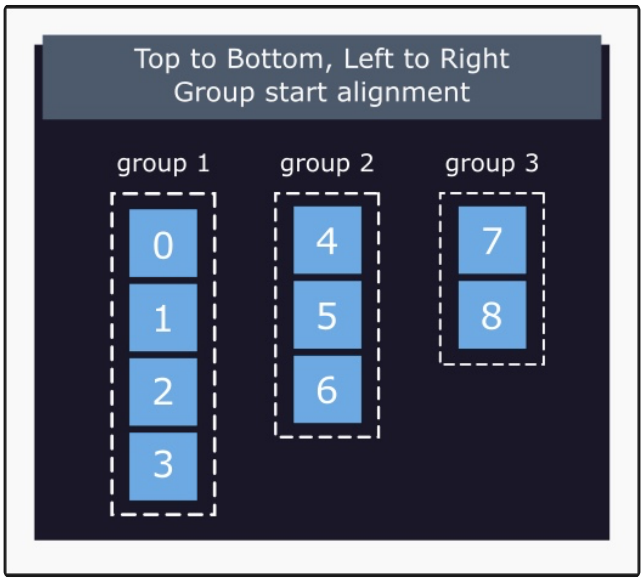


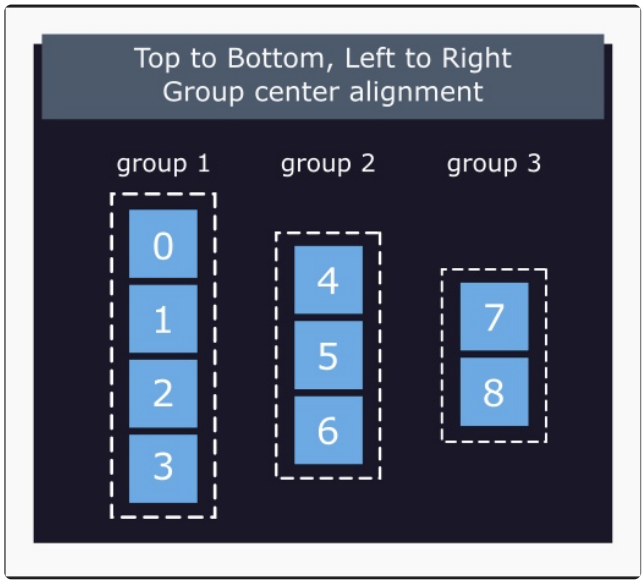




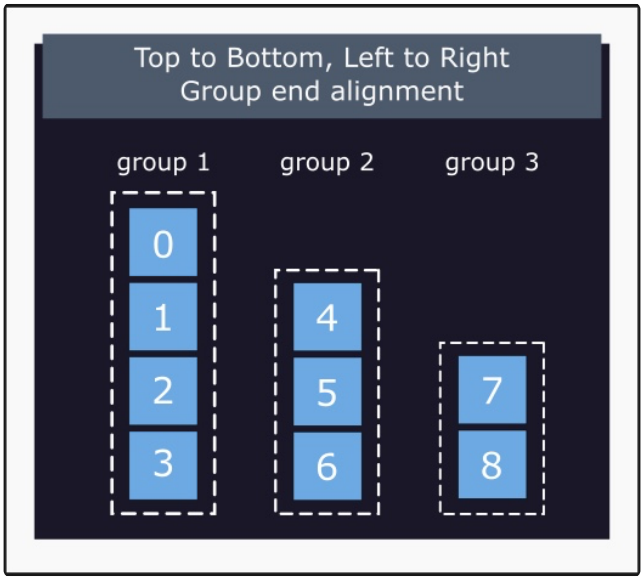


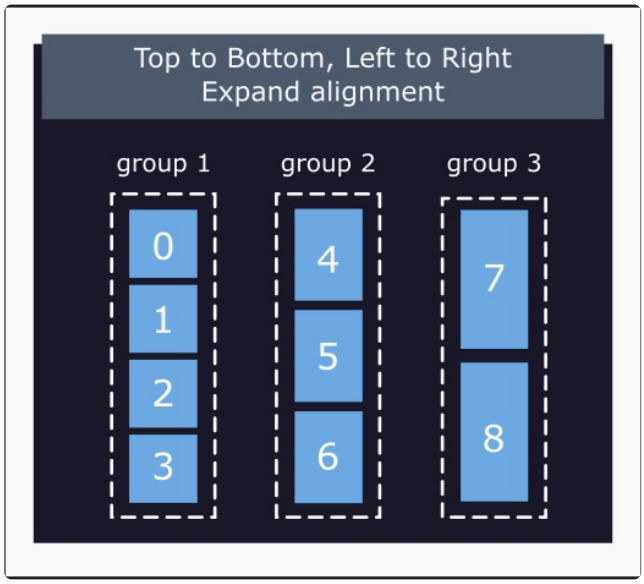
Top to Bottom, Left to Right (Columns)



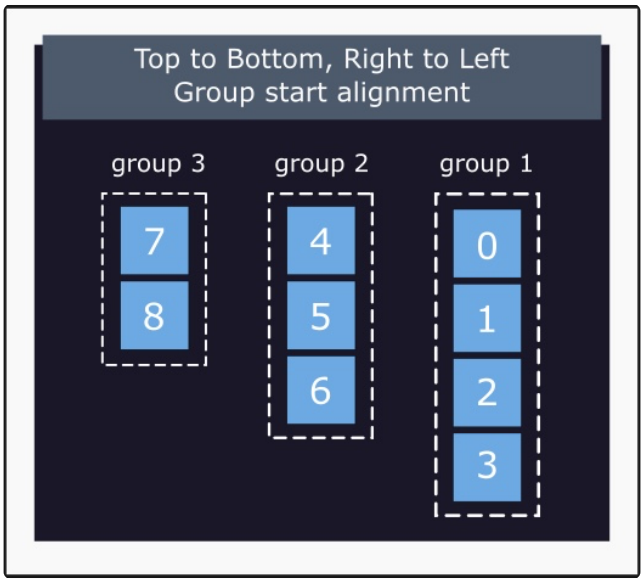


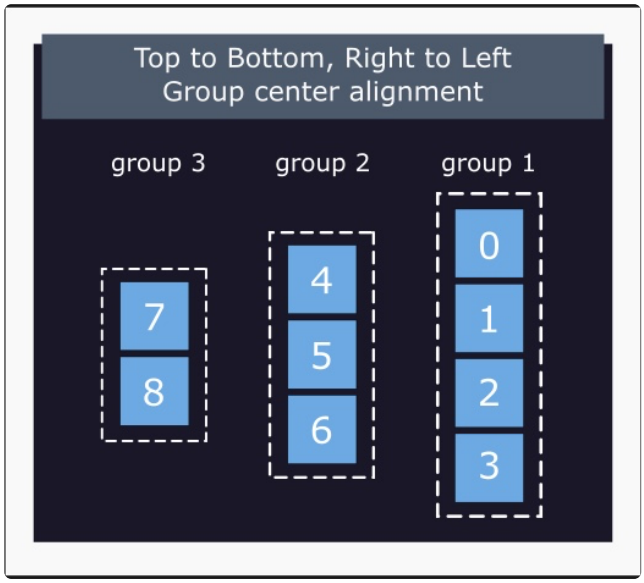
created with craft

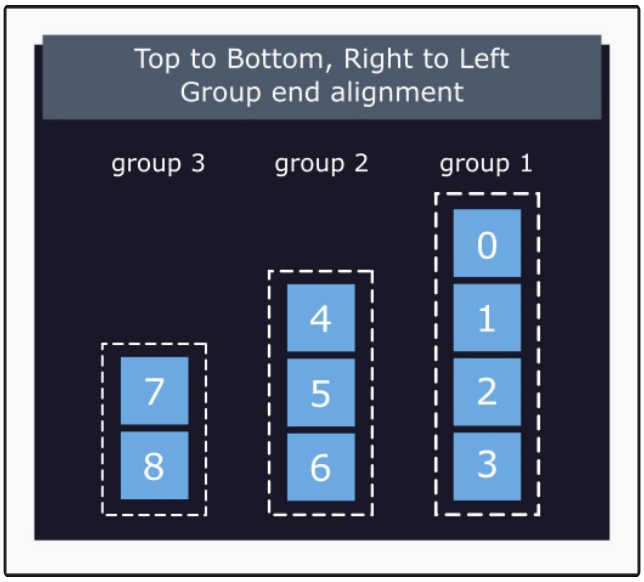


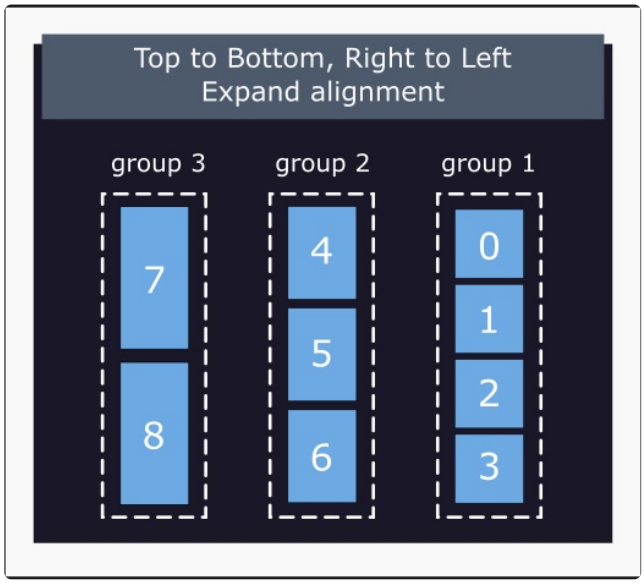


Top to Bottom, Right to Left (Columns)



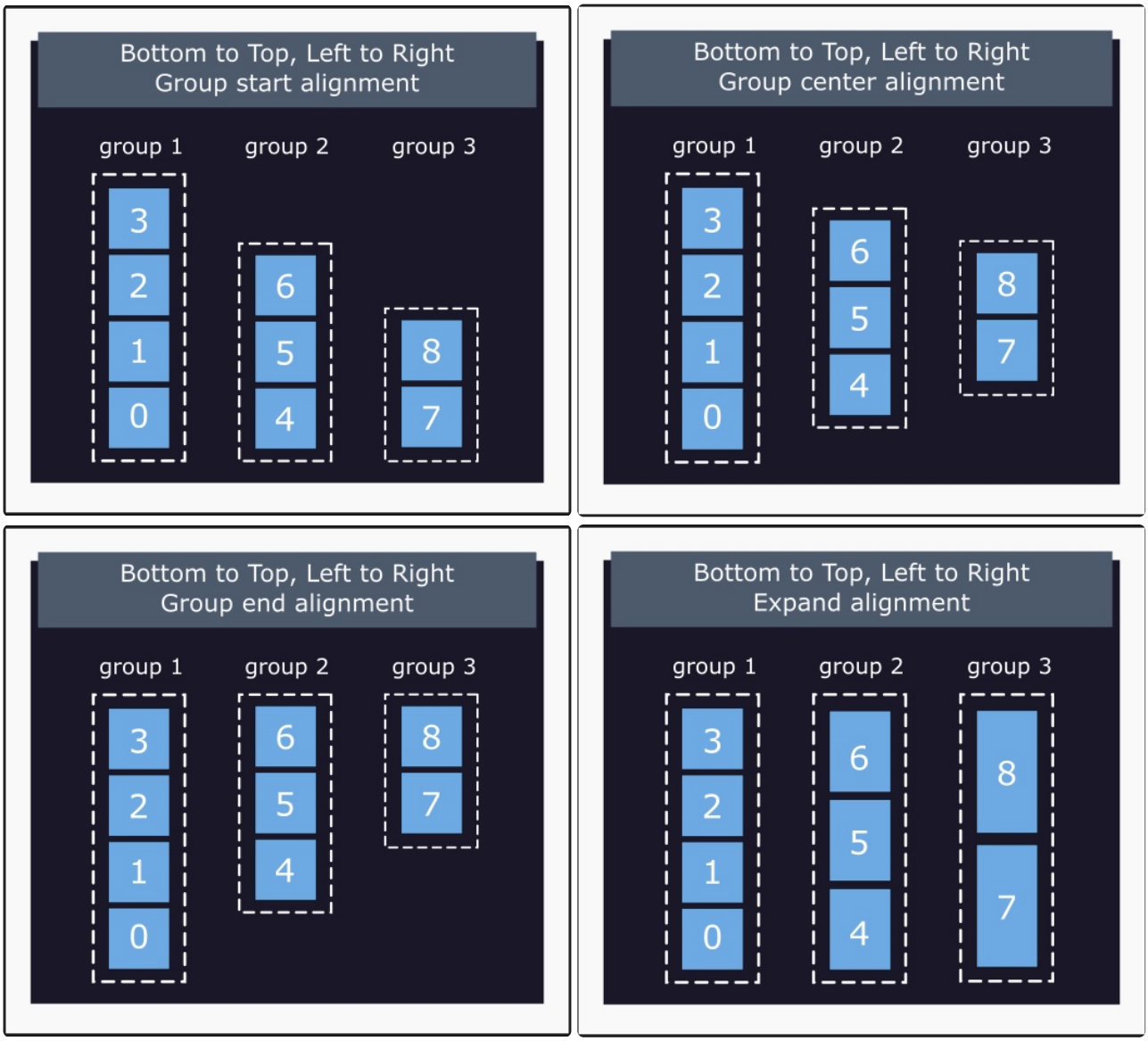






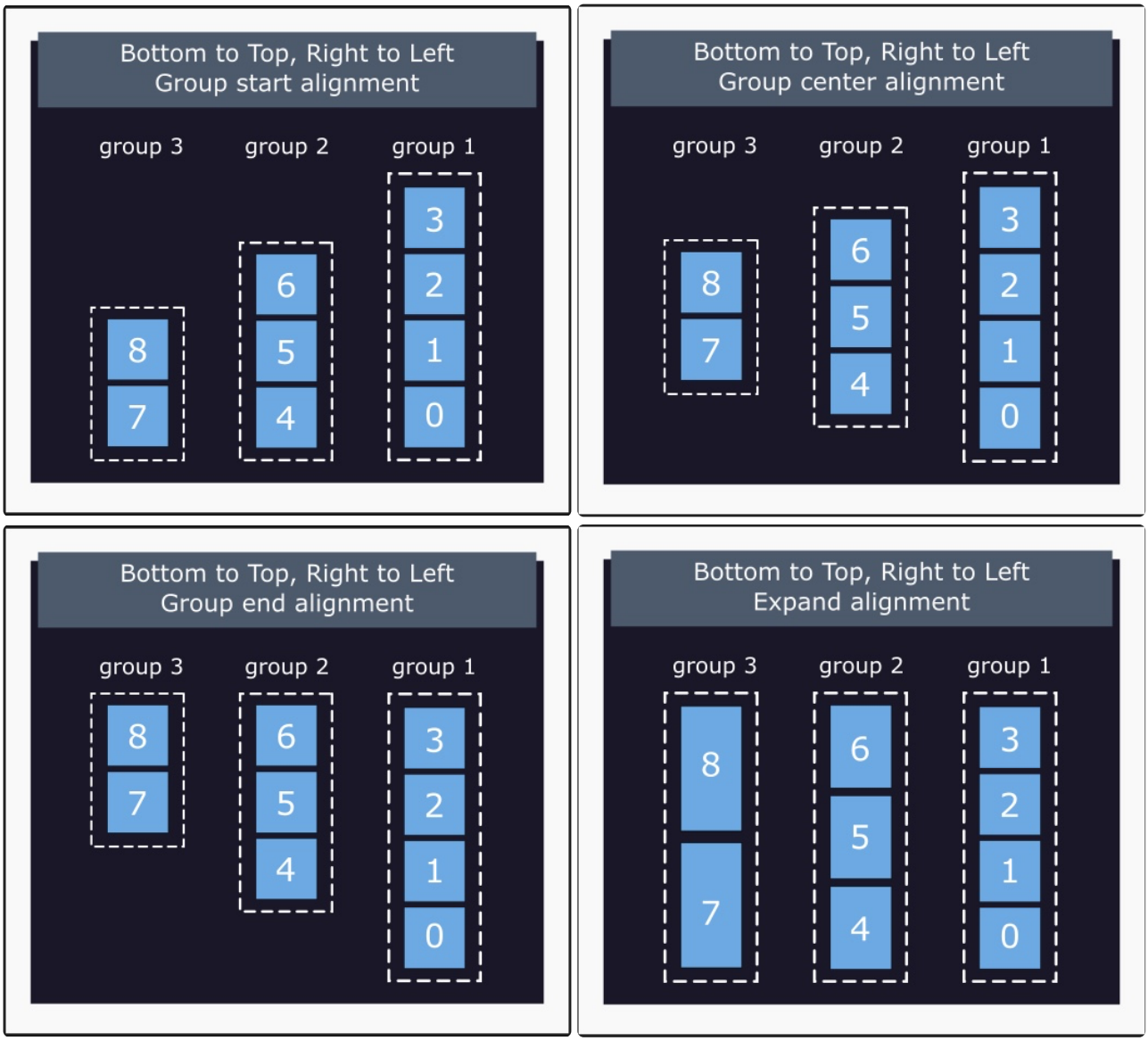
created with craft

Bottom to Top, Left to Right (Columns)



Bottom to Top, Right to Left (Columns)

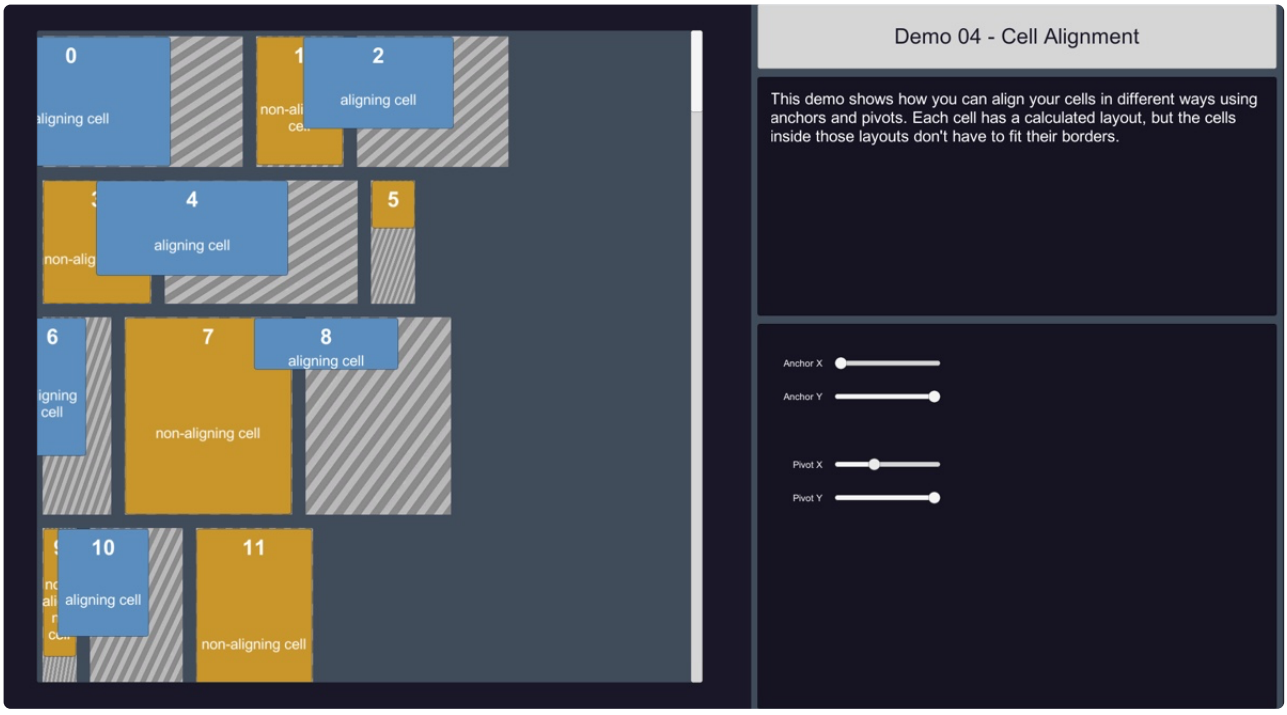
created with craft



Layout & Alignment

Cell layouts are calculated to fit to the largest cell size of a group (row or column). Each cell layout will act as a container for your own cell views. You can move and align your cells within these containers freely.

created with craft



Cell alignment demo included with the plugin

Cell Expansion

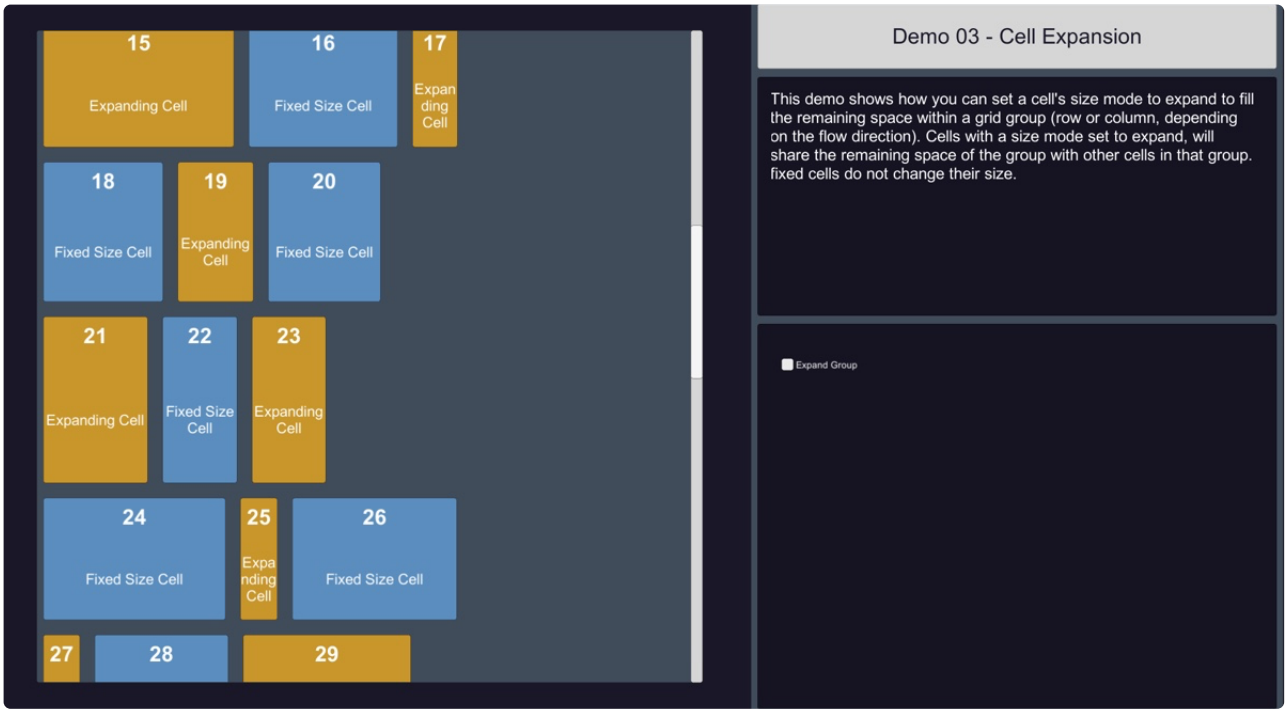
If your Group Alignment is set to Expand, then the cells inside the group will expand to fit the group if their cell property has an expansion weight greater than zero.

All the cells inside the group that have their cell property expansion weight greater than zero will divide the remaining space inside the group based on their expansion weight.

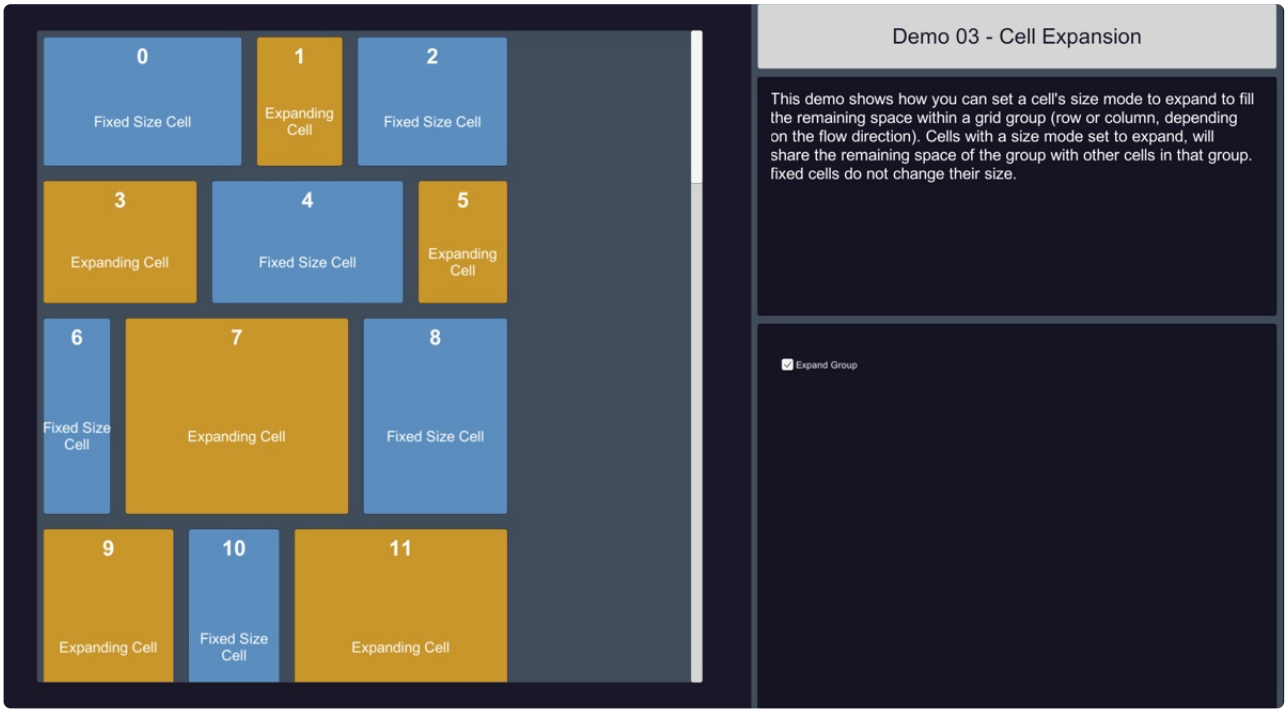
> For example: if a group has three cells and each cell takes 25% of the expansion available, for a total of 75% expansion in that group, then these weights will be adjusted so that each cell takes 33% $(0.25 * (1/0.75))$.
>
> Another example: if a group has two cells, one with 50% and one with 200% expansion weights for a total of 250% expansion, then these values will be normalized to 20% $(0.5 * (1/2.5))$ and 80% $(2 * (1/2.5))$, respectively.

All cells that have an expansion weight of zero will not expand, keeping the minimum size set in the cell properties.

created with craft



Group alignment set to start. Orange cells have their expansion weights greater than zero.



Group alignment set to expand. Orange cells have their expansion weights greater than zero.

created with craft

↑

Recycling

EnhancedGrid has the option to recycle cells into an object pool. This cuts down on the number of objects that are present in memory, displaying only the cells that are visible. As the grid is scrolled, it will determine if cells have fallen outside the viewable area, and if so, return them to the object pool. When cells come back into view, they are pulled from the object pool if possible. If no cells exist in the object pool, then a new cell is created.

If your grid only has a small number of cells or all the cells are constantly visible, turning off recycling will actually improve performance since the grid will not have to calculate what cells are visible as it is scrolled.

Cell Life Cycle

Each cell goes through the following life cycle, calling the listed delegates, informing any listeners of the event:

Cell Created. Called when a cell is newly formed, not when it is reused.

1. gridCellCreated
2. gridCellActivated

Cell Reused. Called when a cell is reused from the object pool, not when it is first created. Only relevant when recycling.

1. gridCellVisibilityChanged
2. gridCellReused
3. gridCellActivated

Cell Recycled. Called when a cell is being sent back to the object pool. Only relevant when recycling.

1. gridCellWillRecycle
2. gridCellVisibilityChanged
3. gridCellDidRecycle

Viewable Area

The grid will determine what cells are visible based on the viewable area as the grid is scrolled. The viewable area is the Viewport rect, plus any extended visible area specified. Some advantages of having an extended visible area include:

- You can check less often if new cells have become visible / invisible, cutting down on the processing time while scrolling.
- You can preload information into cells. Useful if you have asynchronous methods of retrieving data (like downloading images from the cloud).

created with craft

> If you have the extended visible area set to something greater than zero, then you can take advantage of delayed checking for cell visibility by also setting the MinimumScrollForActiveCellUpdate values. Keep x values lower than the ExtendVisibleAreaLeft and ExtendVisibleAreaRight, and the y values lower than the ExtendVisibleAreaTop and ExtendVisibleAreaBottom. This will reduce the chance of a cell "popping" into or out of view where the user can see it.

Occlusion

EnhancedGrid uses an occlusion graph to quickly determine what cells are visible while recycling. Instead of traversing the entire list of cell layouts to see which ones fall in the viewable area, it instead partitions the Viewport into quadrants, known as occlusion sectors. Each of these sectors can have sub-sectors, depending on the occlusion depth you set on the grid.

Cell layouts are placed into these sectore so that EnhancedGrid can ignore cells that are not visible in their parent sectors, cutting down on the number of checks the grid has to do.

Occlusion Depth

You don't want to set the occlusion depth too high as each depth creates an exponential amount of sectors. Large sets of data will want to use a higher depth value, where small data sets would want smaller depth values. The number of occlusion sectors created is four to the power of the depth.

- Depth 1: $4^1 = 4$
- Depth 2: $4^2 = 16$
- Depth 3: $4^3 = 64$
- Depth 4: $4^5 = 256$
- etc.

Setting the depth too high will actually cause more work to be done to determine a cell's visibility for smaller data sets.

You can debug the occlusion sectors by turning on the Show Occlusion Sectors property of the grid. This should not be used in production, only for getting a sense of how the sectors are being created.

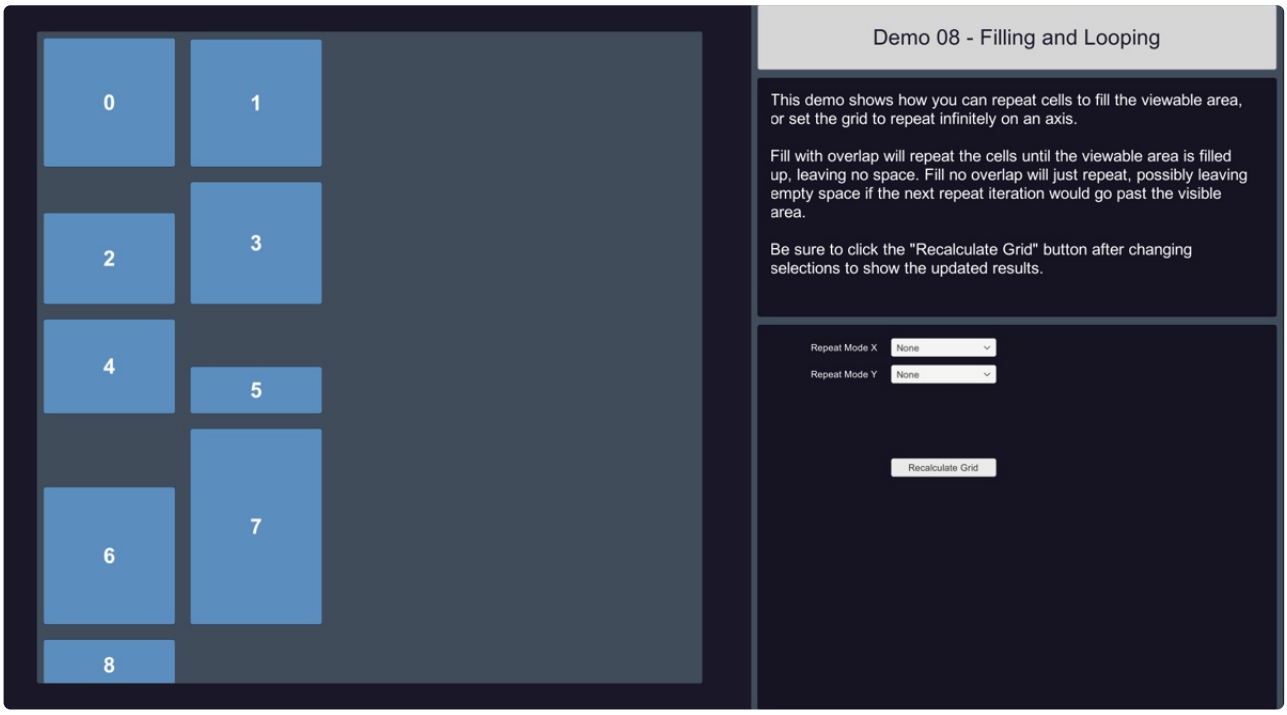
created with craft

↑

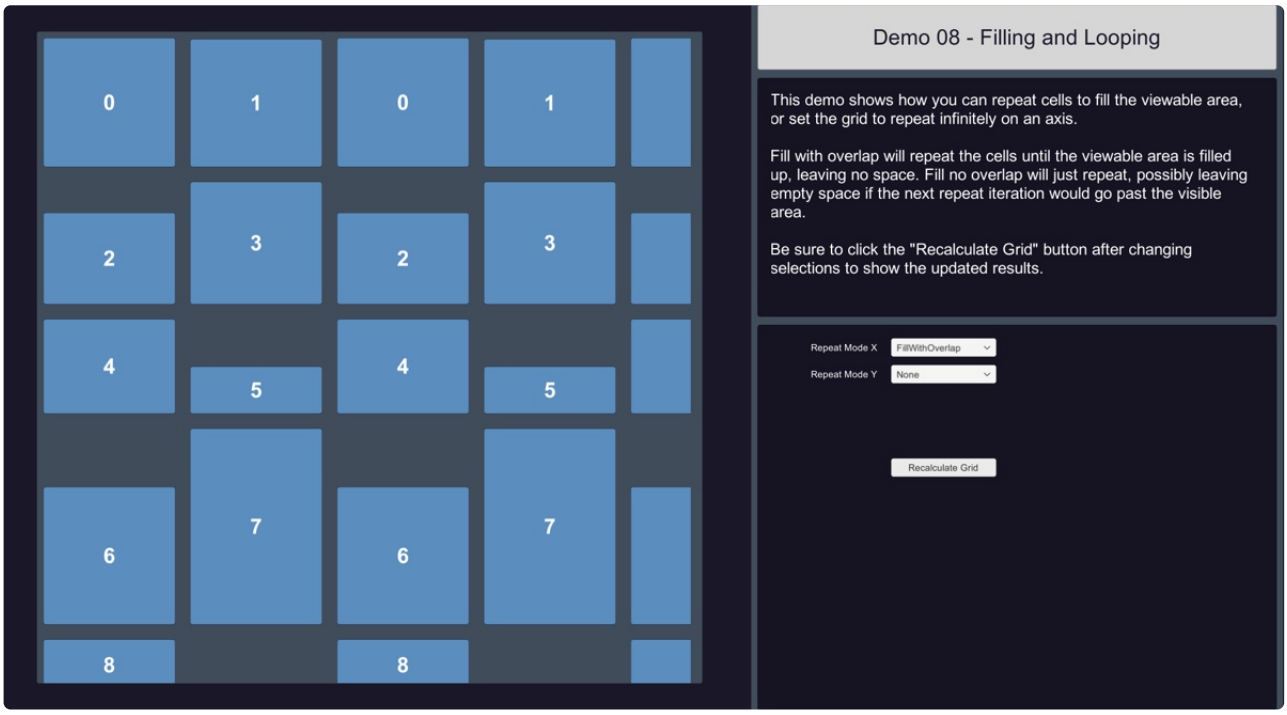
Filling & Looping

EnhancedGrid has four repeat modes which can be set on a per-axis basis. So you can have different repeat modes for each of the x and y directions.

- None: cells are not repeated or looped

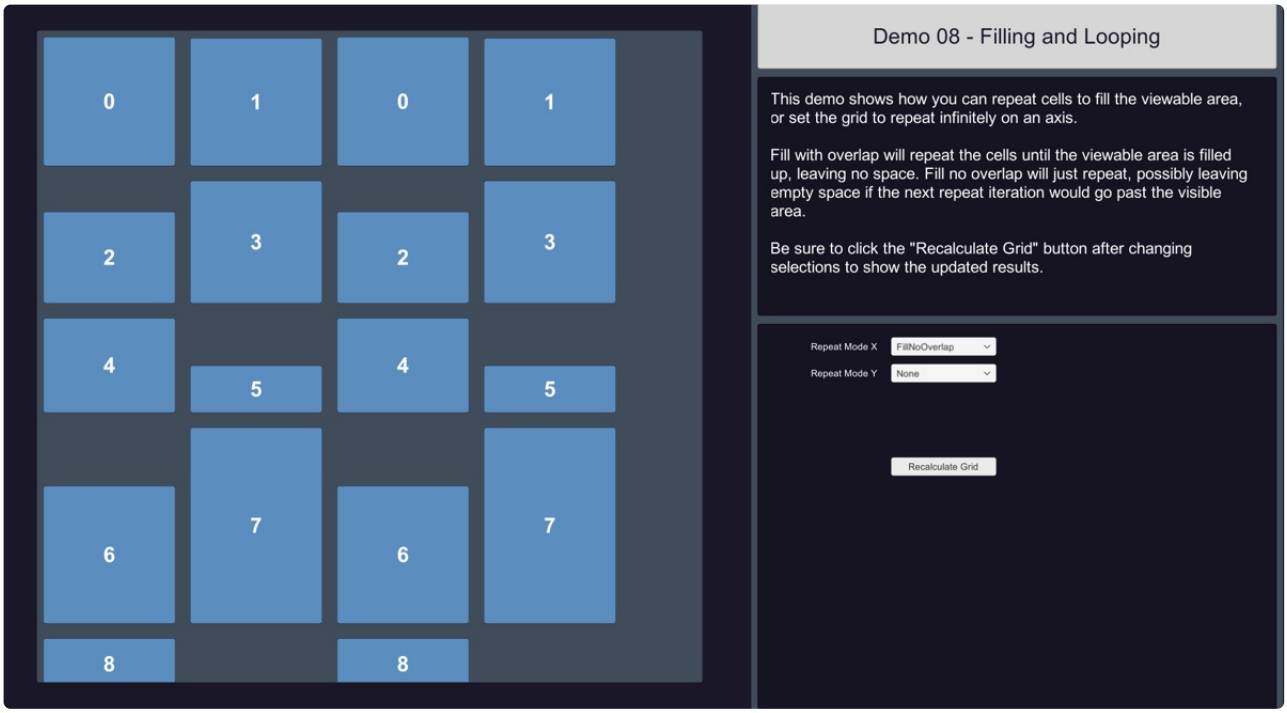


- Fill with Overlap: cells will be repeated to fill up the remaining viewport space along the axis. Overlap allows the repeated cells to go passed the viewport boundaries.

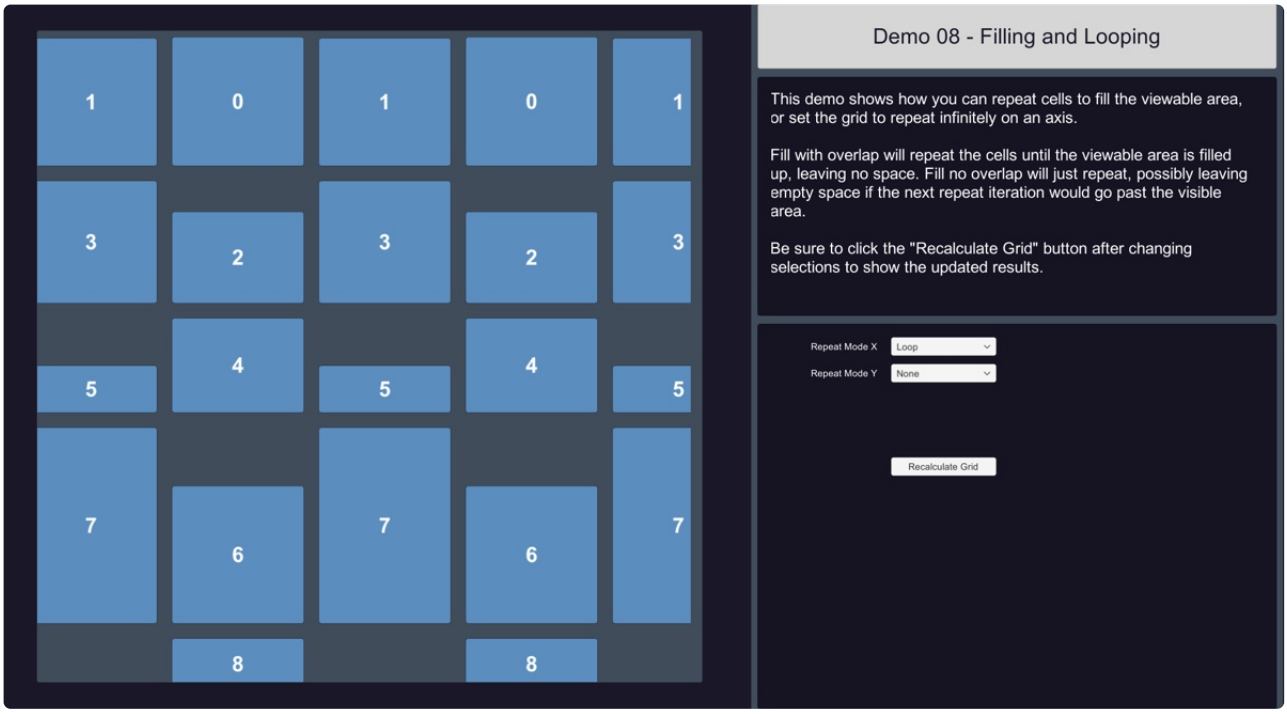


created with craft

- Fill No Overlap: Cells will be repeated to fill up the remaining viewport space along the axis. If the repeated cells go past the viewport boundaries, they will not be used. Depending on how



- Loop: Cells will be repeated to fill the viewport boundaries and three copies of this will be used to simulate an infinite list of cells. The grid will jump backward and forward to these copies, depending on the scroll direction, to give a seamless simulation of infinity.



created with craft

↑

Jumping

Jumping is when you move to a different position or cell in a grid. It is like jumping to a different spot on a game board. You can choose how you want to move to the new spot, like sliding or bouncing. This is called tweening. Tweening is the way the movement looks, like how smooth or bumpy it is. Tween settings are per axis, so you can transition differently along the x and y directions.

There are different ways to jump to a new spot in the grid. One way is to jump to a spot based on the position of a cell at a specific data index. Another way is to jump to a spot based on how far away it is from where you are now. You can also jump to a spot and make sure the cell is fully visible on the screen. It's like moving around on a map.

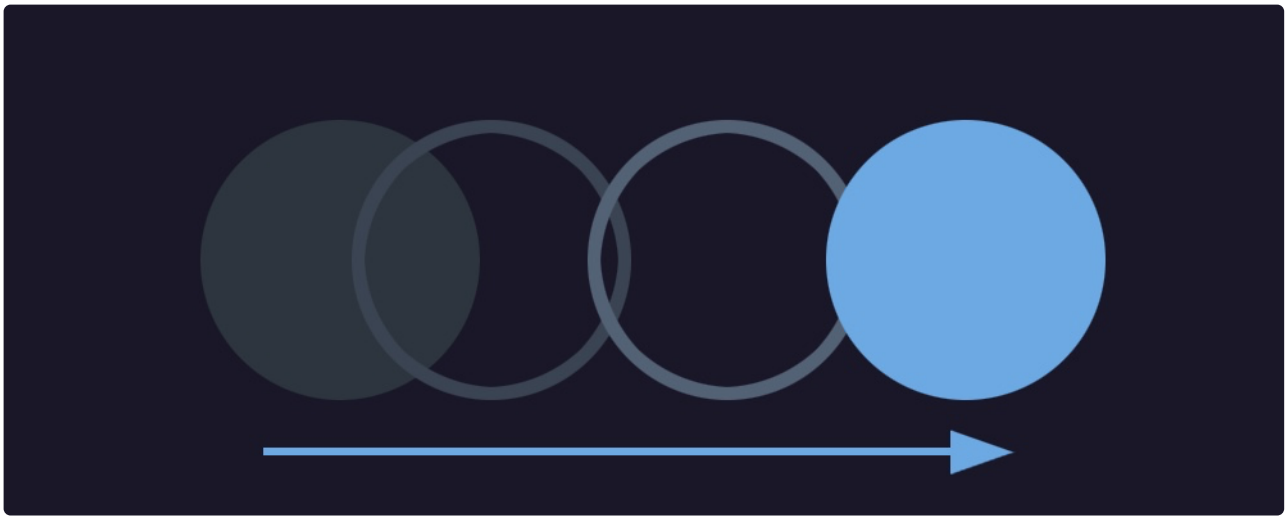
There are several methods to move to different locations within the grid:

JumpToDataIndex: Jumps to a location based on the position of a cell at a given index.

JumpOffset: Jumps to a position based on an offset from the current position.

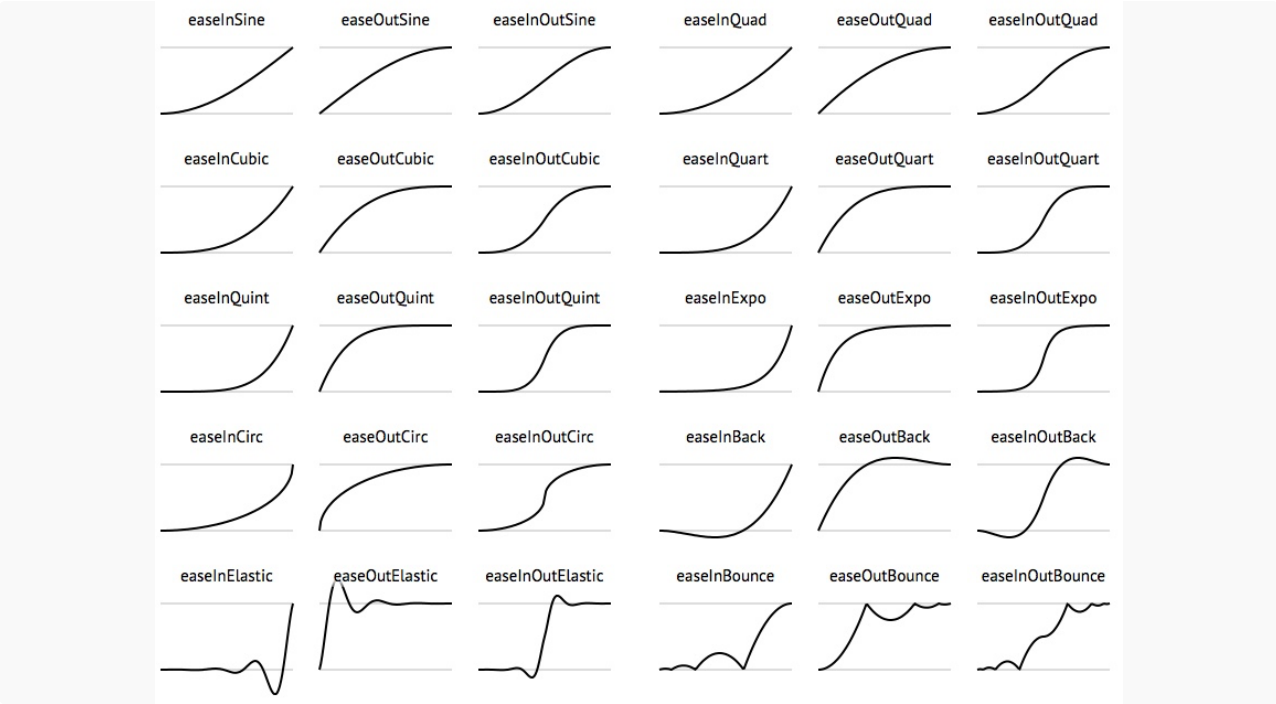
JumpToMakeDataIndexFullyViewable: Jumps to a location based on the position of a cell at a given index, trying to make the cell fully in the visible area. Because some cells may be larger than the viewing area, you can also specify where in the cell to focus on.

Tweening



Tweening, or inbetweening, is the process of changing a value over time based on a mathematical formula. Each axis of the grid can be tweened independently, giving a lot of options on how your jumps perform.

created with craft



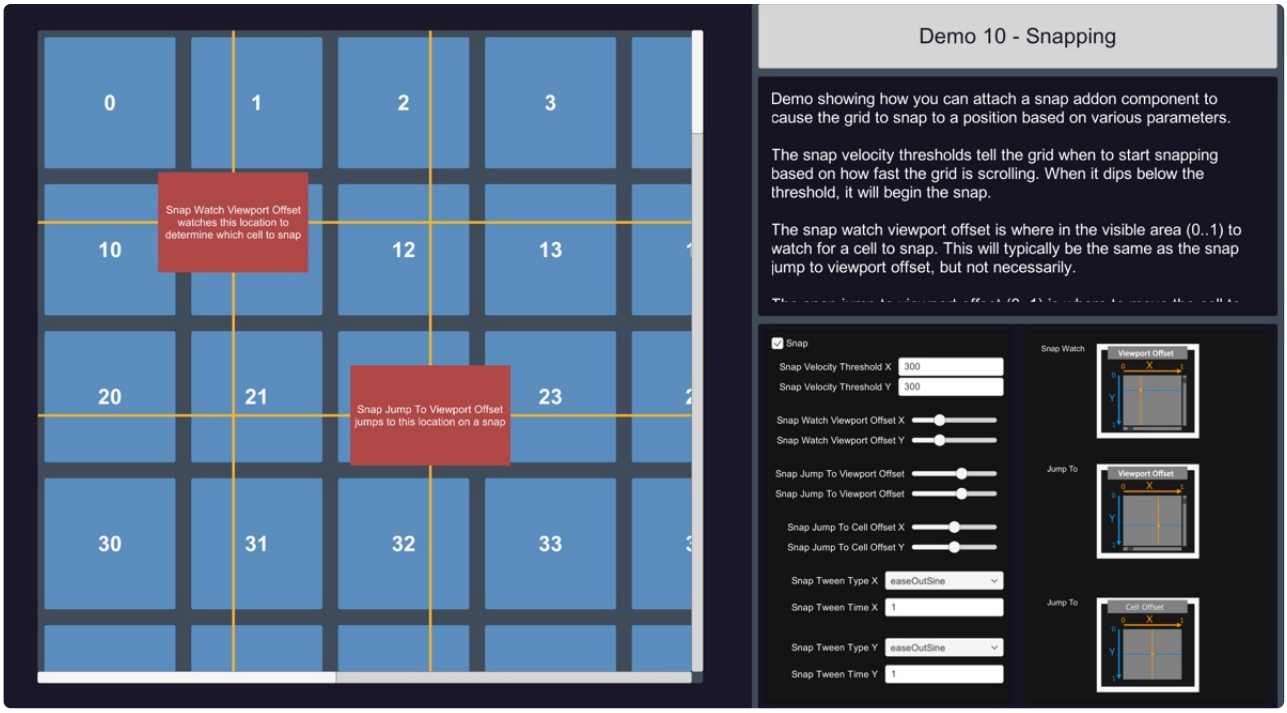
Tweening graphic. Source [inkfood.com]

created with craft

↑

Snapping

EnhancedGrid has additional components that you can attach, allowing you to expand the plugin to give it more features. One of those addons is the Snap component.

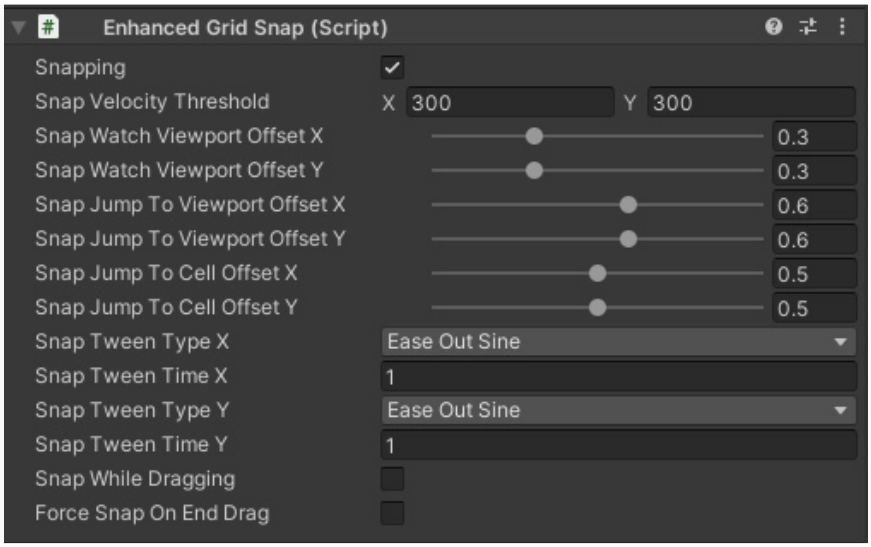


Snapping demo included with the plugin

Snapping lets you automatically move to a cell once the grid has slowed enough. This can be useful to always keep a cell in particular location.

To use snapping, you must add the snap component to the same game object as the EnhancedGrid component. Be sure snap is enabled in the component.

created with craft



Snapping component inspector

Snap Velocity Threshold: The maximum speed of the grid before calling the snap process. The speed of the grid must be below both the x and y thresholds for snapping to occur.

Snap Watch Viewport Offset: Where in the viewport to look for the cell to snap on. This value is between zero and one for each axis.

Snap Jump To Viewport Offset: Where in the viewport to move the cell to after the snap. This value is between zero and one for each axis.

Snap Jump To Cell Offset: Where in the cell to center on. This value is between zero and one on each axis.

Snap Tween Type: The Tween function to use to transition to the new location. Tweening is by axis, so you can have different methods for the x and y directions.

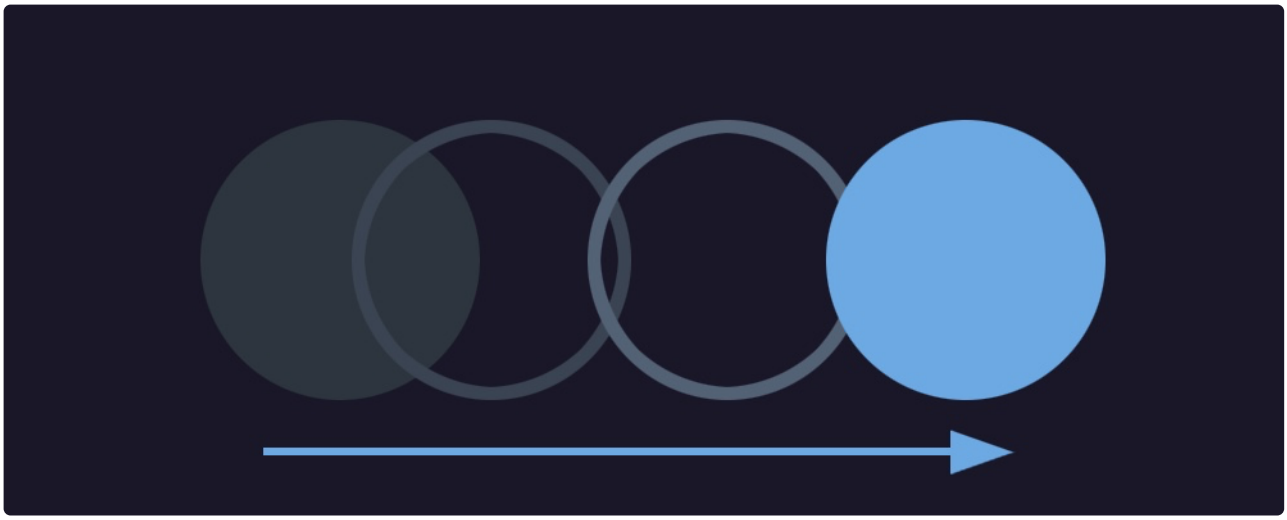
Snap Tween Time: The duration of the snap on each axis.

Snap While Dragging: Whether to call the snap if the pointer (mouse or finger) is currently dragging the grid. Typically this would not be preferred as it would give a jittering motion.

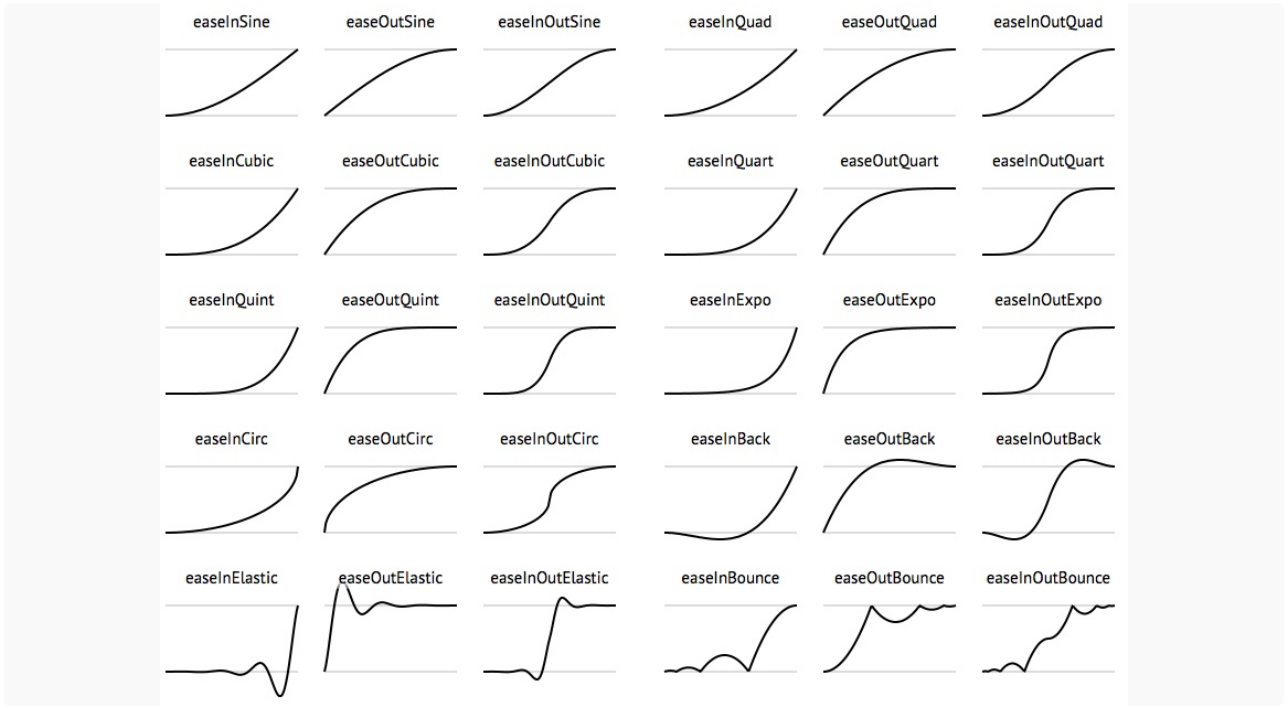
Force Snap On End Drag: When dragging by a pointer (mouse or finger) ends, always snap if this is enabled. This is useful if the drag has no velocity when it ends, but you still want to call the snap function.

Tweening

created with craft



Tweening, or inbetweening, is the process of changing a value over time based on a mathematical formula. Each axis of the grid can be tweened independently, giving a lot of options on how your snaps perform.



Tweening graphic. Source inkfood.com

created with craft